

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Procedurální generování dungeonů ve 2D hrách

Implementace a srovnání algoritmů v herním enginu Godot



2026

Vedoucí práce:
doc. RNDr. Jan Konečný, Ph.D.

Jesse Sadowý

Studijní program: Informační technologie,
kombinovaná forma

Bibliografické údaje

Autor:	Jesse Sadowý
Název práce:	Procedurální generování dungeonů ve 2D hrách (Implementace a srovnání algoritmů v herním enginu Godot)
Typ práce:	bakalářská práce
Pracoviště:	Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby:	2026
Studijní program:	Informační technologie, kombinovaná forma
Vedoucí práce:	doc. RNDr. Jan Konečný, Ph.D.
Počet stran:	35
Přílohy:	test
Jazyk práce:	český

Bibliographic info

Author:	Jesse Sadowý
Title:	Procedural Generation of Dungeons in 2D Games (Implementation and Comparison of Algorithms in Godot Game Engine)
Thesis type:	bachelor thesis
Department:	Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense:	2026
Study program:	Information Technologies, combined form
Supervisor:	doc. RNDr. Jan Konečný, Ph.D.
Page count:	35
Supplements:	electronic data in the storage of department of computer science
Thesis language:	Czech

Anotace

10-20 řádků Bakalářská práce se zabývá procedurálním generováním dungeonů ve 2D hrách. Práce zkoumá nejen vybrané algoritmy, které se v této tématice často používají, ale zároveň je i porovnává z hlediska efektivity, výkonu, pokrytí a možnostech využití. V práci je také popsána samotná implementace a grafická ukázka v herním enginu Godot.

Synopsis

Bachelor's thesis focuses on procedural generation of dungeons in 2D games. It explores various methods and techniques used for generation and subsequently compares their efficiency and performance within the Godot game engine environment.

Klíčová slova: procedurální generování; dungeon; 2D hry; Godot; herní engine, algoritmus

Keywords: procedural generation; dungeon; 2D games; Godot; game engine, algorithm

Děkuji, děkuji, děkuji.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	8
2	Současný stav řešené problematiky	8
2.1	Procedurální generování obsahu ve hrách	8
2.1.1	Vymezení a definice pojmu	8
2.1.2	Nevýhody procedurálního generování	9
2.1.3	Přístupy k procedurálnímu generování	9
2.1.4	Využití procedurálního generování v herním průmyslu	10
2.2	Roguelike hry a jejich vztah k procedurálnímu generování	11
2.2.1	Beneath Apple Manor	11
2.2.2	Rogue	11
2.2.3	Elite	12
2.3	Generování dungeonů ve 2D hrách	13
2.3.1	Požadavky na generování dungeonových map	13
2.4	Přehled metod generování	14
2.4.1	Náhodné umístění místností a chodeb	14
2.4.2	Binary Space Partitioning	15
2.4.3	Metoda náhodné procházky	16
2.4.4	Buněčné automaty	16
2.4.5	Metoda bludiště	17
2.4.6	Další metody	18
2.5	Shrnutí způsobů generování	18
3	Experimentální část	19
3.1	Použité technologie	19
3.1.1	Herní engine Godot	19
3.1.2	Skriptovací jazyk GDScript	19
3.2	Návrh systému generování	20
3.2.1	Reprezentace mapy	20
3.2.2	Struktura generátoru	21
3.2.3	Vizualizace mapy a hráče	22
3.2.4	Ovládání generování a zobrazení statistik	23
3.3	Implementace generovacích metod	24
3.3.1	Náhodné generování místností a chodeb	24
3.3.2	Binary Space Partitioning	25
3.3.3	Metoda náhodné procházky	26
3.3.4	Buněčné automaty	27
3.3.5	Metoda bludiště	28
4	Výsledky	29
4.1	Porovnání výsledných map	29
4.2	Výpočetní náročnost metod	29
4.3	Praktické využití výsledků	29

5	Diskuse	29
5.1	Silné a slabé stránky jednotlivých metod	29
5.2	Zkušenosti z implementace	29
5.3	Možnosti dalšího rozšíření práce	29
	Závěr	30
	Conclusions	31
A	Ukázky zdrojového kódu	32
B	Odkaz na repozitář projektu	32
C	Zkratky	32
D	Obsah elektronických dat	32
	Seznam zkratk	34
	Literatura	35

Seznam tabulek

1	Srovnání vybraných metod generování dungeonových map	19
2	Ukázka reprezentace mapy v mřížce.	20

1 Úvod

Procedurální generování obsahu představuje v současném herním vývoji významný přístup, který umožňuje automatizovaně vytvářet části herního světa na základě předem definovaných pravidel a algoritmů. V oblasti digitálních her se tento princip uplatňuje zejména tam, kde je žádoucí vysoká variabilita prostředí, znovuhratelnost nebo snížení nároků na ruční tvorbu rozsáhlého množství obsahu. Jednou z typických oblastí využití je generování dungeonových map, které tvoří důležitou součást 2D her, zejména titulů inspirovaných žánrem roguelike.

Dungeonové mapy kladou na generování specifické požadavky. Nestačí, aby byly pouze náhodné nebo vizuálně rozmanité, ale musí zároveň splňovat základní podmínky hratelnosti a vhodné rozmístění jednotlivých prvků prostředí. Volba generovací metody proto výrazně ovlivňuje nejen výsledný vzhled mapy, ale i herní zážitek hráče.

Cílem této bakalářské práce je představit vybrané metody procedurálního generování dungeonových map ve 2D hrách, analyzovat jejich vlastnosti a implementovat jejich praktické ukázky v herním engineu Godot. Součástí práce je návrh systému v podobě jednoduché hry, který umožní jednotlivé algoritmy realizovat v jednotném prostředí, vizualizovat jejich výstupy a následně je mezi sebou porovnat a ověřit specifické aspekty důležité právě pro generovaný obsah. Důraz je kladen jak na teoretické vymezení problematiky, tak na praktickou implementaci a zhodnocení výsledků.

Práce je rozdělena do několika kapitol. Nejprve jsou popsány základní principy procedurálního generování, jeho přínosy, omezení a historický vývoj. Další kapitola se věnuje samotnému generování dungeonů ve 2D hrách, požadavkům na výsledné mapy a přehledu vybraných metod. Poté je představen návrh prototypu aplikace a použité technologie. V dalších částech práce je popsána implementace jednotlivých algoritmů, jejich vyhodnocení a vzájemné porovnání. Závěr práce shrnuje dosažené výsledky, očekávání a možné rozšíření.

2 Současný stav řešené problematiky

2.1 Procedurální generování obsahu ve hrách

2.1.1 Vymezení a definice pojmu

Procedurální generování ve hrách lze vymezit jako algoritmickou tvorbu herního obsahu s omezeným nebo nepřímým zásahem člověka. Tento přístup umožňuje automatizovaně vytvářet různé prvky hry bez nutnosti jejich ruční přípravy. Herní obsah přitom zahrnuje široké spektrum složek, například mapy, úrovně, pravidla, úkoly, zbraně či předměty případně i samotné vlastnosti jednotlivých předmětů. PCG se tedy neomezuje pouze na herní prostředí, ale může být aplikováno i na další části herního systému. V závislosti na konkrétním návrhu může ovlivňovat nejen prostorovou strukturu hry, ale také její mechaniky, obtížnost či

samotný průběh hraní.

V současnosti je pojem generování obsahu často spojován s metodami umělé inteligence, tato práce se však zaměřuje výhradně na algoritmické postupy založené na předem definovaných pravidlech. Nezabývá se tedy adaptivním ani učícím se chováním systémů umělé inteligence.

Pojem hra je v odborné literatuře obtížné jednoznačně vymezit, protože zahrnuje široké spektrum forem lišících se cíli, pravidly, mírou interaktivity i použitou platformou. V kontextu této práce je proto termín hra chápán jako digitální videohra realizovaná v reálném čase, ve které hráč interaguje s virtuálním prostředím přes definované herní mechaniky. Pro účely dalšího textu se práce zaměřuje především na 2D videohry určené pro osobní počítače, případně další běžné platformy.

Termíny „procedurální“ a „generování“ zdůrazňují, že jde o proces realizovaný pomocí počítačových procedur a algoritmů. Člověk nastavuje vstupní podmínky a spouští samotný proces, výsledkem je však automaticky vytvořený herní obsah, který není pevně předem daný, ale vzniká až za běhu programu.

V následujících kapitolách se podíváme na PCG více do hloubky.

2.1.2 Nevýhody procedurálního generování

Přestože procedurální generování obsahu přináší řadu výhod, jeho využití s sebou nese také několik významných omezení a úskalí.

Jedním z hlavních problémů je nižší míra kontroly nad výsledným obsahem. U ručně navržených úrovní má vývojář plnou kontrolu nad tempem hry, rozmístěním výzev, předmětů i klíčových herních momentů. Naproti tomu u procedurálního generování je výsledná podoba světa závislá na implementovaném algoritmu a zvolených parametrech. To může vést k nevyváženým úrovním, nelogickému rozmístění prvků nebo situacím, které jsou příliš jednoduché či naopak extrémně náročné. Vývojář proto musí věnovat značné úsilí návrhu, ladění a testování generátoru, aby bylo dosaženo uspokojivé kvality výsledného obsahu.

Dalším rizikem je možnost vzniku repetitivního nebo uměle působícího obsahu. I přes to, že je obsah generován automaticky a náhodně, často vychází z omezené množiny předem definovaných prvků a pravidel. Pokud není systém navržen dostatečně variabilně, může hráč postupně rozpoznat opakující se vzorce, což může negativně ovlivnit herní zážitek. Tento problém je sice možné minimalizovat vhodným návrhem algoritmu a důkladným laděním, nicméně zcela eliminovat jej bývá obtížné.

2.1.3 Přístupy k procedurálnímu generování

Na procedurální generování se můžeme podívat z různých úhlů pohledu a lze ho rozdělit dle určitých specifik, jako je například jak systém reaguje na hráče případně zdali využíváme konkrétní vstupní parametry.

Vzhledem k chování hráče se můžeme v praxi nejčastěji setkat s adaptivním či neadaptivním přístupem. V případě neadaptivního (statického) generování

je obsah vytvářen bez ohledu na chování hráče a jeho průběh samotnou hrou. Všechny generované struktury vycházejí ze stejných pravidel a reakce hráče na podněty neovlivní výsledek. Naopak adaptivní generování pracuje s informacemi o hráči. Systém může sledovat úspěšnost hráče, rychlost reakcí, délku průchodu úrovně, využívané herní prvky nebo způsob řešení situací a na základě těchto dat dále upravovat generovaný obsah.

Dalším důležitým rozdělením je deterministický a stochastický přístup. Deterministické generování vychází z pevně daných počátečních hodnot (například seed) a při stejném vstupu vždy produkuje stejný výstup. Tento přístup umožňuje reprodukovatelnost výsledků, což je výhodné například při testování nebo sdílení konkrétních map mezi hráči. Stochastické generování naproti tomu pracuje s náhodností bez pevně stanoveného počátečního stavu, takže výsledky se budou lišit i při opakovaném spuštění algoritmu.

Z hlediska samotného procesu generování lze rozlišit konstruktivní metody a přístup označovaný jako generate-and-test. Konstruktivní metody vytvářejí obsah postupně podle předem definovaných pravidel a omezení. Každý krok generování přímo přispívá k vytvoření výsledné struktury. Naproti tomu metoda generate-and-test nejprve vytvoří potenciální řešení, které je následně ověřeno podle stanovených kritérií. Pokud výsledek nevyhovuje, proces se opakuje. Tento přístup může být výpočetně náročnější, ale umožňuje lépe kontrolovat výsledné vlastnosti mapy.

V literatuře se rovněž objevuje pojem mixed-authorship (smíšené autorství), kdy se na tvorbě obsahu podílí jak algoritmus, tak lidský návrhář. Systém může například generovat návrhy úrovně, které následně autor upravuje, případně může reagovat na částečně ručně vytvořenou strukturu. Tento přístup kombinuje kreativitu člověka s rychlostí automatizovaného generování.

Plně automatická generace naopak probíhá bez zásahu člověka během samotného vytváření obsahu. Algoritmus pracuje pouze na základě vstupních parametrů. Tento způsob je typický zejména pro roguelike hry a další tituly, u nichž je cílem vysoká variabilita jednotlivých průchodů.

2.1.4 Využití procedurálního generování v herním průmyslu

S procedurálním generováním se dnes setkáváme téměř v každé moderní videohře, jeho počátky však sahají již do 70. let 20. století. V tomto období byl výpočetní výkon i dostupná operační paměť velmi omezené. Tyto limity výrazně omezovaly množství ručně vytvářeného obsahu, a vývojáři proto hledali způsoby, jak vytvářet variabilní herní prostředí s minimálními nároky na paměť RAM. Právě v tomto kontextu se začaly prosazovat procedurální přístupy.

2.2 Roguelike hry a jejich vztah k procedurálnímu generování

V kontextu PCG je nutné zmínit také pojem roguelike, který označuje herní subžánr úzce spojený s náhodně generovaným obsahem. Název žánru je odvozen od hry Rogue, jež výrazně ovlivnila podobu mnoha pozdějších titulů.

Roguelike hry bývají typické náhodně generovanými úrovněmi, tahovým systémem, permanentní smrtí postavy (permadeath) a zpravidla i vyšší obtížností. Náhodné generování zde neplní pouze technickou funkci, ale představuje zásadní designový prvek, který podporuje průzkum, strategické rozhodování i adaptaci hráče na proměnlivé prostředí. Právě díky těmto vlastnostem se roguelike hry staly jedním z nejvýraznějších příkladů praktického a systematického využití procedurálního generování v herním průmyslu.

2.2.1 Beneath Apple Manor

Jednou z prvních her, které implementovaly PCG, byla Beneath Apple Manor z roku 1978. Autorem titulu byl Don Worth a hru vydalo studio The Software Factory. Primárně byla vyvíjena pro počítač Apple II.

Herní prostředí v Beneath Apple Manor bylo generováno algoritmicky při každém novém spuštění hry. Díky tomu nebylo rozložení místností, chodeb ani jednotlivých herních prvků či nepřátel nikdy zcela totožné, a právě díky tomu byla hra přívětivá pro znovuhratelnost. Cílem hry bylo získat zlaté jablko a postup bylo možné ukládat, takže po smrti hráče se načetla uložená úroveň znovu, přičemž hráč přišel o část svých atributů.

Autor hry čerpal inspiraci z programu Dragon Maze, který rovněž využíval náhodné výpočty ke generování bludišť. Tento princip však dále rozvinul a navrhl algoritmus vytvářející funkční a hratelnou strukturu dungeonů složenou z místností propojených chodbami. Hra navíc umožňovala hráči ovlivnit některé parametry generování, například počet místností na jednotlivých patrech, barevnou variantu zobrazení nebo obtížnost. Po vytvoření struktury úrovně algoritmus náhodně rozmístil také nepřátele, poklady a další herní objekty. S rostoucí hloubkou dungeonů se zároveň zvyšovala obtížnost hry, zejména síla nepřátel.

Použitý algoritmický přístup lze zařadit mezi rané příklady způsobu generování založených na místnostech a chodbách, které se později stalo jedním ze základních modelů využívaných v hrách typu roguelike. Beneath Apple Manor tak představuje významný historický příklad využití těchto metod, jehož principy byly dále rozvíjeny v následujících dekáдах.

2.2.2 Rogue

Za zmínku stojí také hra Rogue, která si oproti výše zmíněnému Beneath Apple Manor získala výrazně větší popularitu a měla zásadní vliv na další vývoj herního designu. Právě podle této hry vznikl podžánr označovaný jako roguelike, který jsme si popsali výše, a z jejích principů vychází dodnes.

Hra byla představena v roce 1980 a jejími autory jsou Michael Toy, Glenn Wichman a Ken Arnold. Původně byla vyvíjena pro unixové systémy a zobrazovala herní svět pomocí ASCII znaků v textovém terminálu. Jednotlivé prvky prostředí byly reprezentovány znaky, například hráč symbolem @, stěny znakem #, nepřátelé různými písmeny abecedy. Tento způsob zobrazení nebyl pouze důsledkem technických omezení tehdejšího hardware, ale zároveň umožňoval poměrně flexibilní práci s herním světem a jeho generováním.

Rogue bývá popisována jako jedna z prvních roguelike her, která výrazně zpopularizovala princip permanentní smrti (permadeath). Postup hrou nebylo možné ukládat a jakmile hráč zemřel, musel začít zcela od začátku. Hráči tak zůstávaly pouze nabyté zkušenosti a znalost herních mechanik. Smrt zde proto nepředstavovala pouze neúspěch, ale také nedílnou součást učení se hernímu systému.

Cílem hry je najít Amulet of Yendor, který se nachází v nejhlubším patře dungeonů, a následně se vrátit zpět na povrch. Dungeon je tvořen několika patry (tradičně 26), přičemž každé z nich je generováno procedurálně. Patra se skládají z místností propojených chodbami a jejich rozložení se při každém novém průchodu mění. Náhodně jsou rozmístěni také nepřátelé, pasti, poklady, zbraně a další předměty.

Prvek náhody se projevuje i ve vlastnostech předmětů. Například lektvary, svitky či prsteny mají při každé nové hře odlišné označení a jejich účinek není hráči předem znám. I pokud hráč nalezne stejný typ předmětu jako v předchozí hře, jeho vlastnosti se mohou lišit. Tím je posílena nejistota a nutnost experimentování.

Kombinace procedurálně generovaných úrovní a permanentní smrti zajišťuje, že každý průchod hrou je unikátní. Hráč se tak nemůže spoléhat na zapamatování konkrétní mapy, ale musí se přizpůsobovat aktuálně vygenerovaným podmínkám. Právě tento princip se stal jedním ze základních stavebních kamenů celého roguelike žánru [1, 2].

2.2.3 Elite

Dalším významným titulem využívajícím procedurální generování je hra Elite z roku 1984, kterou vytvořili David Braben a Ian Bell. Na rozdíl od předchozích her se nejedná o dungeon, ale o vesmírný simulátor obchodování a soubojů. Přesto je Elite z hlediska procedurálního generování velmi důležitým milníkem.

Hra byla původně vydána pro počítač BBC Micro, který disponoval velmi omezenou pamětí. Z tohoto důvodu nebylo možné ukládat rozsáhlý herní svět přímo v paměti. Vývojáři proto zvolili přístup, kdy byl herní svět generován algoritmicky na základě předem definovaného číselného základu (seed). Pomocí tohoto deterministického postupu bylo možné při každém spuštění znovu vytvořit stejnou galaxii bez nutnosti jejího explicitního uložení.

Elite obsahovalo osm galaxií, přičemž každá z nich zahrnovala stovky hvězdných systémů. Vlastnosti jednotlivých systémů, jako je jejich název, ekonomika, úroveň technologického rozvoje či typ vlády, byly generovány pomocí pseudo-

náhodných algoritmů. Procedurálně byly generovány také ceny komodit a další herní parametry.

Tento přístup ukázal, že procedurální generování není omezeno pouze na tvorbu dungeonů nebo jednotlivých úrovní, ale může být využito i pro vytvoření rozsáhlých herních světů. Elite tak demonstrovalo, že algoritmická generace obsahu představuje efektivní řešení zejména v podmínkách omezených výpočetních a paměťových zdrojů [3].

2.3 Generování dungeonů ve 2D hrách

V předchozí kapitole byly popsány obecné principy procedurálního generování obsahu a jeho využití v herním průmyslu. Tato kapitola se zaměřuje konkrétně na generování dungeonových map ve 2D hrách, které představují jednu z nejčastějších aplikací procedurálního přístupu.

Dungeonové mapy mají svá specifika, která je odlišují od jiných typů herních prostředí. Typicky se skládají z místností a chodeb, obsahují jasně definované průchozí a neprůchozí oblasti a musí splňovat určité požadavky na hratelnost, jako je průchodnost celé mapy nebo vhodné rozmístění herních prvků. Generování takových struktur proto vyžaduje algoritmy, které dokáží zajistit nejen variabilitu výsledku, ale i splnění základních strukturálních vlastností.

V této kapitole budou nejprve definovány požadavky kladené na generování dungeonových map. Následně budou představeny vybrané metody generování, které se v praxi používají, a které tvoří teoretický základ pro implementaci popsanou v dalších kapitolách.

2.3.1 Požadavky na generování dungeonových map

Při návrhu algoritmu pro generování dungeonové mapy je nutné zohlednit několik základních požadavků, které vycházejí jak z technických omezení, tak z hlediska hratelnosti. Generovaná mapa by neměla být pouze náhodným shlukem místností a chodeb, ale strukturovaným prostředím, které umožňuje smysluplný průchod a interakci.

Jedním ze základních požadavků je průchodnost mapy neboli connectivity. Všechny klíčové části mapy by měly být vzájemně dosažitelné, aby nedocházelo k izolovaným oblastem bez možnosti přístupu. Tento požadavek je zásadní zejména u her, kde je cílem projít celou úroveň nebo nalézt konkrétní objekt.

Dalším důležitým aspektem je kontrola struktury a rozložení prostoru. Mapa by měla obsahovat přiměřený poměr otevřených a uzavřených prostor, vhodnou velikost místností a dostatečně široké průchody, aby hráč mohl bez problémů projít. Příliš husté nebo naopak příliš prázdné prostředí může negativně ovlivnit hratelnost i orientaci hráče.

Významným požadavkem je rovněž variabilita výsledků. Opakované spuštění generátoru by mělo vést k odlišným mapám, aby byl zachován prvek znovuhratelnosti. Zároveň je však vhodné, aby generování respektovalo určitá pravidla nebo omezení, která zajistí konzistentní kvalitu výsledku.

Z technického hlediska je důležitá také výpočetní náročnost algoritmu. Generování by mělo probíhat v přiměřeném čase a nemělo by výrazně zatěžovat systém, zejména pokud je realizováno za běhu hry (tzv. online generování). U jednodušších 2D dungeonových her je obvykle možné využít algoritmy s relativně nízkou časovou složitostí.

V neposlední řadě je vhodné umožnit parametrizaci generátoru. Možnost nastavit velikost mapy, počet místností, hustotu chodeb nebo další vlastnosti umožňuje lépe přizpůsobit výsledné prostředí konkrétním požadavkům hry a zároveň usnadňuje testování různých konfigurací [4, 5].

2.4 Přehled metod generování

Na základě výše uvedených požadavků a pravidel lze identifikovat několik přístupů a algoritmů generování dungeonových map. Tyto metody se liší zejména způsobem konstrukce prostoru, mírou kontroly, výslednou strukturou a výpočetní náročností. Některé algoritmy vytvářejí mapu postupně přidáváním jednotlivých prvků, jiné pracují s rozdělováním prostoru nebo s pravidly aplikovanými na mřížku buněk.

Ve 2D hrách se nejčastěji setkáváme s metodami založenými na náhodném umísťování místností a jejich následném propojování, s rozdělováním velkého prostoru na menší části nebo s algoritmy založenými na takzvané náhodné procházce. Každý z těchto přístupů má své výhody i omezení a je vhodný pro odlišný typ herního prostředí.

Cílem této podkapitoly je stručně představit princip fungování vybraných metod, jejich základní charakteristiky a typické oblasti využití. Detailnější rozpracování a samotná implementace vybraných algoritmů budou popsány v následujících kapitolách.

2.4.1 Náhodné umístění místností a chodeb

Generování pomocí náhodného umístění místností a chodeb patří mezi nejpoužívanější a zároveň nejjednodušší přístupy z hlediska implementace.

Základní princip spočívá v tom, že se v rámci mapy nejprve generují místnosti různých velikostí, obvykle obdélníkového nebo čtvercového tvaru. Během generování se kontroluje, aby se nově vznikající místnost nepřekrývala s již existujícími. Výsledkem je množina místností reprezentovaných například souřadnicemi a rozměry. Následně jsou jednotlivé místnosti propojeny pomocí chodeb. Ty bývají realizovány jako jednoduché horizontální a vertikální průchody spojující středy dvou místností nebo jejich nejbližší body. Častým řešením je také propojení ve tvaru písmene L, kdy jsou dvě místnosti spojeny dvojicí kolmých chodeb.

Pro zajištění průchodnosti mapy a dosažitelnosti všech místností se často využívají algoritmy z oblasti teorie grafů. Jedním z typických řešení je použití minimální kostry grafu (Minimum Spanning Tree), která zaručuje propojení všech místností při minimálním počtu chodeb.

Proces generování lze obecně rozdělit do několika kroků:

1. generování náhodných místností v rámci mapy,
2. kontrola jejich překrývání,
3. výběr místností určených k propojení,
4. generování chodeb mezi jednotlivými místnostmi.

Výhodou této metody je především její jednoduchost. Generované dungeony obvykle obsahují přehledně oddělené místnosti a rozumitelnou síť propojení. Nevýhodou může být menší variabilita, protože místnosti mají často pravidelný tvar obdélníků a čtverců, a prostředí tak může působit příliš uniformně. Dalším problémem bývá méně efektivní využití prostoru, kdy mezi jednotlivými místnostmi zůstávají větší nevyužité oblasti. Komplikace může přinášet i samotné propojování. U jednodušších implementací může docházet ke kolizím chodeb, nevhodnému křížení nebo ke vzniku příliš dlouhých průchodů, které narušují dynamiku pohybu v prostředí.

Z těchto důvodů bývá metoda často kombinována s dalšími postupy nebo doplněna o dodatečná pravidla, která omezují velikosti a vzájemné vzdálenosti místností, délku chodeb nebo celkové rozložení prostoru [4, 5].

2.4.2 Binary Space Partitioning

Metoda Binary Space Partitioning (BSP) patří mezi přístupy založené na systematickém rozdělování prostoru. Základní princip spočívá v tom, že se celá mapa nejprve reprezentuje jako jeden velký obdélníkový nebo čtvercový prostor, který je následně postupně rozdělován na menší části. Každé rozdělení může probíhat horizontálně nebo vertikálně a výsledkem je hierarchická struktura podobná binárnímu stromu.

Samotný proces generování obvykle začíná rozdělením celé mapy na dvě části. Tyto části se dále rekurzivně dělí, dokud nedosáhnou určité minimální velikosti. Každé rozdělení vytváří dva nové uzly ve stromové struktuře, přičemž listové uzly představují konečné oblasti, ve kterých mohou být vytvořeny místnosti. Takto vzniklá stromová reprezentace umožňuje přehledně uchovávat strukturu rozdělení prostoru.

V jednotlivých vzniklých oblastech jsou následně vytvořeny místnosti, jejichž velikost je o něco menší než velikost dané oblasti. Tím vzniká přirozený prostor mezi místnostmi, který může být využit například pro generování chodeb nebo dalších herních prvků. Po vytvoření místností je nutné zajistit jejich propojení. To se obvykle provádí postupným propojováním místností mezi sousedními částmi stromové struktury, často od nejnižších úrovní stromu směrem nahoru.

Díky tomuto postupu lze relativně snadno zajistit vzájemnou dosažitelnost všech místností. Zásadní výhodou BSP je poměrně dobrá kontrola nad velikostí a rozmístěním jednotlivých částí mapy. Parametry dělení prostoru umožňují ovlivnit výsledný vzhled prostředí a předejít situacím, kdy by vznikaly příliš malé nebo naopak příliš velké místnosti.

Nevýhodou této metody může být opět poměrně pravidelný a místy uniformní vzhled generovaného prostředí, protože výsledná struktura často odráží samotný způsob dělení prostoru. Mapy vytvořené pomocí BSP tak mohou působit méně přirozeně než mapy generované například pomocí buněčných automatů [4, 5].

2.4.3 Metoda náhodné procházky

Metoda náhodné procházky, často označovaná také jako *Drunkard's Walk*, je založena na jednoduchém principu postupného rozšiřování prostoru pomocí náhodného pohybu. Algoritmus začíná na určité počáteční pozici v mapě a v každém kroku se náhodně přesune do jednoho ze sousedních polí. Směr pohybu je obvykle vybírán ze čtyř základních směrů v mřížce, tedy nahoru, dolů, doleva nebo doprava.

Při každém kroku je aktuální buňka označena jako průchozí prostor. Postupným opakováním tohoto procesu vzniká síť chodeb a otevřených prostor, jejichž tvar závisí na délce a průběhu náhodné procházky. Výsledná struktura mapy bývá velmi nepravidelná a rozdílná při každém generování.

Proces generování obvykle pokračuje po předem stanovený počet kroků nebo dokud není dosaženo určitého poměru průchozích buněk v mapě. V některých variantách algoritmu může být použito více nezávislých „agentů“, kteří se po mapě pohybují současně a vytvářejí tak komplexnější strukturu prostředí.

Výhodou této metody je velmi jednoduchá implementace a schopnost generovat přirozeně působící struktury. Nevýhodou je naopak menší kontrola nad výsledným tvarem mapy a obtížnější zajištění rovnoměrného využití prostoru. V některých případech může docházet k tomu, že většina průchozích oblastí vzniká v blízkosti počáteční pozice algoritmu, zatímco jiné části mapy zůstávají téměř nevyužité.

Podobně jako u buněčných automatů se tato metoda často používá při generování jeskynních nebo jinak přirozeně působících prostředí [5].

2.4.4 Buněčné automaty

Buněčné automaty představují přístup k procedurálnímu generování založený na aplikaci lokálních pravidel na mřížku buněk. Mapa je v tomto případě reprezentována jako dvourozměrná mřížka, kde každá buňka může být například ve stavu stěna nebo průchozí prostor. Na začátku generování je mapa obvykle inicializována náhodným rozdělením těchto stavů, což vytváří základní strukturu prostředí.

V následujících iteracích se stav jednotlivých buněk aktualizuje na základě stavu jejich sousedních buněk. Nejčastěji se používá tzv. Mooreovo sousedství, které zahrnuje osm buněk obklopujících aktuální buňku. Pravidla generování mohou například určovat, že buňka zůstane průchozí pouze tehdy, pokud má dostatečný počet průchozích sousedů, případně že se změní na stěnu, pokud je obklopena větším množstvím stěn.

Opakovaným aplikováním těchto pravidel dochází k postupnému vyhlazování náhodně vytvořené struktury. Izolované buňky nebo malé skupiny buněk se postupně odstraňují a vznikají větší souvislé oblasti. Výsledkem jsou mapy s nepravidelnými tvary, které často připomínají přirozené jeskynní systémy.

Výhodou této metody je schopnost generovat organicky působící prostředí s nepravidelnými tvary, které se liší od klasických dungeonů založených na místnostech a chodbách. Nevýhodou může být menší kontrola nad přesnou strukturou mapy a také nutnost dodatečných kroků, které zajistí průchodnost celé mapy. V některých případech je proto nutné odstranit izolované oblasti nebo dodatečně propojit jednotlivé části prostředí [4, 5].

2.4.5 Metoda bludiště

Metody generování bludišť vytvářejí struktury tvořené převážně úzkými chodbami, které tvoří souvislou síť průchodů. Mapa je v tomto případě obvykle reprezentována jako mřížka buněk oddělených stěnami. Samotné generování spočívá v postupném odstraňování těchto stěn tak, aby vznikla síť propojených průchodů.

Tyto algoritmy často vycházejí z principů grafových algoritmů, například prohledávání do hloubky (Depth-First Search) s návratem (backtracking) nebo algoritmů typu Prim či Kruskal. V případě algoritmu založeného na prohledávání do hloubky se začíná v náhodně zvolené buňce, ze které se postupně prochází do sousedních dosud nenavštívených buněk. Při každém kroku se odstraní stěna mezi aktuální buňkou a vybraným sousedem. Pokud již neexistuje žádný nenavštívený soused, algoritmus se vrací zpět k předchozí buňce.

Primův algoritmus vytváří bludiště postupným rozšiřováním již vytvořené oblasti. Algoritmus začíná v náhodné buňce a postupně vybírá sousední stěny, které oddělují navštívené buňky od nenavštívených. Pokud odstranění vybrané stěny spojí dvě dosud nepropojené oblasti, je tato stěna odstraněna a nová buňka se stane součástí bludiště.

Kruskalův algoritmus naproti tomu pracuje s množinou všech stěn v mapě. Stěny jsou postupně vybírány v náhodném pořadí a odstraňují se pouze tehdy, pokud jejich odstranění nespojí dvě buňky, které již patří do stejné části bludiště. Tím se postupně vytváří souvislá síť průchodů bez vzniku cyklů.

Výsledkem je tzv. perfektní bludiště, ve kterém mezi dvěma body existuje právě jedna cesta. Taková struktura neobsahuje cykly a vytváří spleť chodeb. Tento typ generování je vhodný zejména pro hry zaměřené na průzkum a orientaci v prostoru.

Nevýhodou této metody je omezený výskyt větších otevřených prostor, protože generovaná struktura je tvořena převážně úzkými chodbami. Z tohoto důvodu bývá v dungeonových hrách často kombinována s jinými metodami, které umožňují vytvářet také větší místnosti. V některých případech se proto po vygenerování bludiště odstraňují vybrané stěny, čímž vznikají cykly a otevřenější struktura mapy [4, 5].

2.4.6 Další metody

Kromě výše uvedených přístupů existuje celá řada dalších metod generování dungeonových map. Patří mezi ně například generování založené na grafech, kde je struktura mapy nejprve reprezentována jako graf místností a jejich propojení. Jednotlivé vrcholy grafu představují místnosti a hrany určují jejich vzájemné propojení. Na základě takto vytvořené abstraktní struktury je následně vytvořena konkrétní geometrická podoba mapy.

Další možností je využití procedurálních gramatik, které definují pravidla pro vytváření jednotlivých částí mapy. Generování poté probíhá aplikací těchto pravidel, podobně jako při tvorbě vět v formálních jazycích. Tento přístup umožňuje poměrně přesně řídit strukturu generovaného prostředí a je vhodný zejména pro hry, které vyžadují specifické uspořádání jednotlivých herních prvků.

V literatuře se rovněž objevují přístupy založené na evolučních algoritmech nebo optimalizačních metodách, které se snaží generované mapy postupně vylepšovat podle předem definovaných kritérií. Tyto metody pracují s populací kandidátních map, které jsou postupně upravovány pomocí genetických operací, jako je mutace nebo křížení. Výsledné mapy jsou následně hodnoceny podle zvolených metrik, například průchodnosti, velikosti prostoru nebo rozložení herních prvků.

Modernější přístupy zahrnují také metody založené na splňování různých omezení. Jedná se o takzvanou constraint-based generation nebo algoritmy typu Wave Function Collapse, které generují mapy postupným výběrem kompatibilních prvků z předem definované množiny vzorů. Tyto metody umožňují generovat velmi rozmanité struktury, avšak jejich implementace bývá složitější.

Uvedené přístupy jsou často výpočetně náročnější než základní algoritmy popsané v předchozích podkapitolách, a proto se v praxi využívají spíše při offline generování obsahu nebo při návrhu komplexnějších herních světů [4, 5].

2.5 Shrnutí způsobů generování

V této kapitole byly představeny vybrané metody procedurálního generování dungeonových map. Jednotlivé přístupy se liší především způsobem konstrukce prostoru, mírou kontroly nad výslednou strukturou a typem generovaného prostředí. Zatímco některé metody vytvářejí spíše pravidelné struktury místností a chodeb, jiné generují nepravidelná a organicky působící prostředí.

Základní charakteristiky popsaných metod shrnuje tabulka 1. Podrobnější porovnání implementovaných algoritmů bude provedeno v kapitole věnované jejich vyhodnocení.

Na základě provedené rešerše byly pro praktickou část práce vybrány algoritmy generování pomocí místností a chodeb, Binary Space Partitioning, buněčné automaty, metoda náhodné procházky a metoda generování bludiště. Tyto algoritmy reprezentují různé přístupy k procedurálnímu generování map a díky tomu je lze snadno vzájemně porovnat.

Metoda	Kontrola struktury	Variabilita	Typ prostředí
Místnosti a chodby	vysoká	střední	klasické dungeons
Binary Space Partitioning	vysoká	střední	strukturované mapy
Buněčné automaty	nízká	vysoká	jeskyně
Náhodná procházka	nízká	vysoká	organické struktury
Bludiště	střední	střední	labyrinty

Tabulka 1: Srovnání vybraných metod generování dungeonových map

3 Experimentální část

Tato kapitola se věnuje návrhu systému pro generování dungeonových map a technologiím použitým při implementaci vytvořeného prototypu. Navržený systém slouží k demonstraci a porovnání vybraných metod procedurálního generování popsanych v předchozí kapitole.

Nejprve jsou představeny nástroje využívané při vývoji aplikace. Následně je popsán samotný návrh systému, zejména způsob reprezentace dungeonové mapy, struktura generátoru a způsob vizualizace výsledného prostředí.

3.1 Použité technologie

V této kapitole budou představeny využívané technologie pro implementaci výše zmíněných metod.

3.1.1 Herní engine Godot

Pro vizualizaci generátoru dungeonových map byl zvolen herní engine Godot. Jedná se o open-source engine určený pro vývoj 2D i 3D her. Poskytuje integrované nástroje pro práci s grafikou, fyzikou, scénami i skriptováním, což umožňuje rychlé vytváření a testování herních prototypů.

Významnou vlastností Godotu je podpora dlaždicových map (TileMap), které jsou při vývoji 2D her často využívány a výrazně to usnadňuje samotné grafické ztvárnění. Tento způsob reprezentace prostředí je zároveň vhodný i pro implementaci algoritmů procedurálního generování, protože umožňuje jednoduchou manipulaci s jednotlivými buňkami mapy.

Engine zároveň umožňuje snadné spouštění a testování různých algoritmů v jednom prostředí, což je výhodné zejména při jejich vzájemném porovnávání.

Další výhodou je integrace skriptovacího jazyka GDScript, který umožňuje rychlou tvorbu generovacích algoritmů a jejich následnou vizualizaci v herním prostředí.

3.1.2 Skriptovací jazyk GDScript

Jak již bylo zmíněno výše, pro implementaci algoritmů byl použit skriptovací jazyk GDScript, který je velmi podobný jazyku Python, avšak je plně podporován

právě zvoleným herním enginem.

GDScript umožňuje přímý přístup k objektům herní scény, komponentám enginu i datovým strukturám. Díky tomu bylo možné jednotlivé generovací metody ztvárnit v samostatných skriptech a následně je snadno integrovat do hlavní aplikace.

Další výhodou je možnost rychlého prototypování a průběžných úprav algoritmů během vývoje. Implementované generátory tak mohou být jednoduše spouštěny a testovány přímo v prostředí enginu, což usnadňuje jejich ladění i následné porovnávání výsledků jednotlivých metod.

3.2 Návrh systému generování

Při návrhu samotného systému generování bylo nutné předem stanovit, jaké metody chceme zahrnout, jak bude vypadat mapa, například velikost a případně počet dlaždic a jak bude ztvárněno ovládání samotné hry. V dalších kapitolách bude implementace rozebrána detailněji.

3.2.1 Reprezentace mapy

Všechny implementované metody generování pracují nad společnou datovou reprezentací mapy ve formě dvourozměrného pole. Mapa je v implementaci uložena jako pole řádků, kde každý prvek odpovídá jedné buňce dungeonové mapy. Jednotlivé buňky obsahují celočíselnou hodnotu určující typ dlaždice.

V implementaci jsou rozlišeny tři základní stavy buněk: prázdný prostor, podlaha a stěna. Tyto stavy jsou reprezentovány číselnými hodnotami 0, 1 a 2, které odpovídají prázdné buňce, průchozí podlaze a stěně. Právě tyto hodnoty následně slouží jako základ pro vykreslení odpovídajících dlaždic.

Hodnota 0 se během generování používá především jako výchozí stav mapy, zatímco hodnoty 1 a 2 představují výslednou strukturu dungeonového prostředí.

Příklad jednoduché reprezentace mapy je uveden v tabulce 2. Ukázka ilustruje průchozí oblast (1), která je obklopena stěnami (2), zatímco zbytek mapy tvoří nevyužitý prostor reprezentovaný hodnotou 0.

0	0	0	0	0
0	2	2	2	0
2	2	1	2	2
2	1	1	1	2
2	2	2	2	2

Tabulka 2: Ukázka reprezentace mapy v mřížce.

Velikost mapy je určena dvěma parametry, šířkou a výškou, které jsou předávány generovacím algoritmům při jejich spuštění. Jednotlivé metody následně vytvářejí a upravují strukturu mapy podle svého principu generování.

Zvolený způsob reprezentace umožňuje sjednotit práci všech implementovaných generovacích algoritmů, protože každá metoda vrací výslednou mapu ve

stejném formátu bez ohledu na svůj vnitřní princip. Výsledná mapa tak může být následně jednotným způsobem zpracována při vizualizaci i při vyhodnocení výsledků.

Výhodou zvoleného řešení je především jeho jednoduchost a přehlednost. Přístup k jednotlivým buňkám je realizován pomocí jejich souřadnic, což usnadňuje jak samotné generování mapy, tak i další operace při práci s mapou.

3.2.2 Struktura generátoru

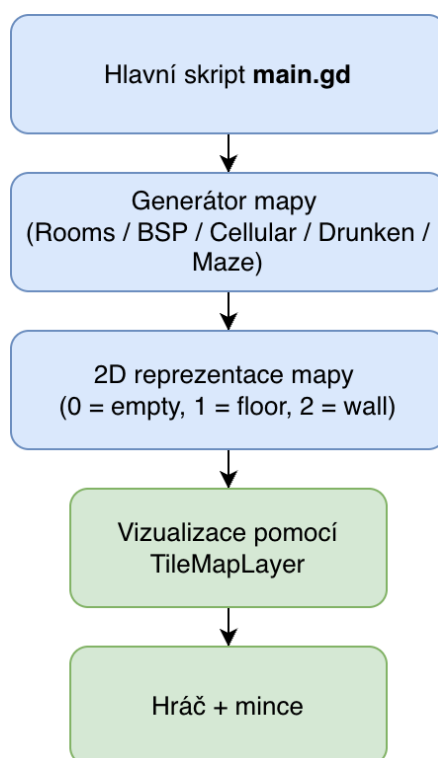
Systém byl navržen modulárně tak, aby bylo možné jednotlivé algoritmy snadno rozšiřovat a vzájemně porovnávat. Každá metoda je tedy implementována v samostatném skriptu a využívá jednotné rozhraní v podobě funkce `generate(width, height)`, která pro zadané rozměry vrací datovou reprezentaci mapy.

Díky tomuto sjednocení lze do aplikace přidávat další generátory bez zásahů do ostatních částí systému. V rámci vytvořeného prototypu byly takto implementovány metody založené na náhodném umístění místností, Binary Space Partitioning, buněčných automatech, náhodné procházce a generování bludiště.

Na úrovni aplikace jejich běh řídí skript `main.gd`, který podle uživatelské volby vybere odpovídající algoritmus, spustí jeho běh a převeze výslednou mapu k dalšímu zpracování. Následně je tato reprezentace předána vrstvě zajišťující vizualizaci a grafické znázornění zvoleného generování je uživateli vykresleno.

Navržená architektura tím odděluje samotnou logiku tvorby mapy od jejího vykreslení. Generátory pracují pouze s daty, zatímco zobrazení výsledku obstarávají samostatné komponenty. Samotné vykreslování tedy neovlivňuje dobu trvání algoritmu, což je žádoucí pro následné porovnávání

Architektura navrženého systému je schematicky znázorněna na obrázku [1](#).



Obrázek 1: Schéma architektury systému generování dungeonových map

3.2.3 Vizualizace mapy a hráče

Pro vizualizaci generovaných map byly v aplikaci použity grafické prvky, takzvané *asset*y. Tyto *asset*y slouží především k přehlednému zobrazení struktury generované mapy a usnadňují orientaci uživatele v herním prostředí. Vizualizace zahrnuje grafiku podlahy, stěn, postavy hráče a sbíratelných objektů.

Zobrazení mapy je realizováno pomocí komponenty `TileMapLayer`. Po dokončení generování je datová reprezentace mapy postupně převedena na odpovídající dlaždice, přičemž jednotlivé typy buněk jsou mapovány na konkrétní grafické prvky představující podlahu nebo stěnu. V zobrazovacím prostředí bylo zvoleno postupné vykreslování mapy po jednotlivých řádcích. Tento způsob sice není nezbytný z hlediska samotného generování, ale usnadňuje vizuální kontrolu výsledků a umožňuje lépe sledovat podobu jednotlivých algoritmů.

Součástí prototypu je také jednoduchá reprezentace hráče, která slouží především k ověření průchodnosti mapy. Hráč je po vygenerování umístěn na vhodnou průchozí pozici tak, aby měl kolem sebe dostatek prostoru a mohl se pohybovat. Je tedy upřednostněna oblast, která poskytuje dostatek volného prostoru v okolí, aby nedocházelo ke kolizi se stěnami ihned po vytvoření mapy.

Pro doplnění testovacího prostředí byly do mapy přidány také sbíratelné objekty ve formě mincí, které jsou po vygenerování náhodně rozmístěny na průchozích dlaždicích. Tyto prvky neslouží jako součást generování samotného, ale usnadňují praktické ověření funkčnosti a pohybu v prostředí a zároveň působí

uživatelsky více přívětivě.

V rámci implementace byl využit volně dostupný balíček *2D Pixel Dungeon Asset Pack*, jehož autorem je grafik vystupující pod jménem Pixel_Poem. Tento balíček obsahuje základní sadu grafických prvků určených pro tvorbu dungeonových roguelike her.

Použité assety mají podobu pixelové grafiky v top-down perspektivě, tedy seshora dolů, a jsou navrženy právě pro práci s dlaždicovými mapami (tilemap), což umožňuje jejich snadnou integraci do herního enginu. Balíček je poskytován jako volně dostupný asset pack, který lze využít v nekomerčních i komerčních projektech.

Vzhledem k tomu, že hlavním cílem práce je implementace a porovnání algoritmů procedurálního generování map, slouží grafická vrstva pouze jako podpůrný nástroj pro vizualizaci a prezentaci výsledků generování.

3.2.4 Ovládání generování a zobrazení statistik

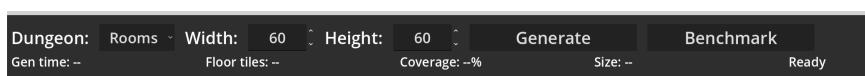
Součástí aplikace je jednoduché uživatelské rozhraní, které umožňuje výběr konkrétní generovací metody a nastavení základních parametrů mapy, zejména její šířky a výšky.

Uživatelské rozhraní obsahuje dvě hlavní ovládací tlačítka, *Generate* a *Benchmark*. Tlačítko *Generate* slouží k vizuálnímu spuštění generování mapy pomocí vybraného algoritmu. Po dokončení generování jsou uživateli zobrazeny základní statistické údaje, které charakterizují vlastnosti vygenerované mapy. Mezi tyto údaje patří zejména čas potřebný pro vytvoření mapy, počet průchozích dlaždic a procentuální pokrytí mapy průchozím prostorem. Generování mapy je zároveň vizualizováno postupným vykreslováním jednotlivých řádků, což umožňuje sledovat průběh tvorby výsledného prostředí.

Tlačítko *Benchmark* slouží k automatizovanému testování implementovaných generovacích metod. Po jeho spuštění aplikace postupně spustí všechny implementované algoritmy pro předem definované velikosti map. V rámci každé konfigurace je generování provedeno opakovaně, aby bylo možné získat reprezentativní hodnoty měřených parametrů. Na rozdíl od běžného generování není při tomto režimu mapa vizualizována, což umožňuje přesnější měření času generování.

Výsledky jednotlivých běhů jsou ukládány do souboru `results.csv`. Pro každý běh algoritmu jsou zaznamenány informace o použité metodě, velikosti mapy, pořadí běhu, čase generování a procentuálním pokrytí mapy průchozí plochou. Tento soubor následně slouží jako vstupní data pro vyhodnocení experimentální části práce, kde jsou výsledky dále analyzovány a graficky znázorněny.

Toto rozhraní slouží především pro účely testování, vizualizace a porovnání implementovaných metod.



Obrázek 2: Uživatelské rozhraní aplikace pro generování map

3.3 Implementace generovacích metod

Principy jednotlivých algoritmů generování map byly popsány v kapitole 2.4. V následujících podkapitolách je popsána jejich konkrétní implementace v rámci vytvořené aplikace.

Každý algoritmus je realizován jako samostatný skript. Jednotlivé skripty můžeme najít v adresáři `res://dungeons/algorithms`. Všechny generátory obsahují funkci `generate(width, height)` obsahující logiku pro daný algoritmus.

Výstupem všech metod je dvourozměrné pole celočíselných hodnot, které reprezentuje typ jednotlivých dlaždic mapy.

Spuštění generování zajišťuje skript `main.gd`, který na základě volby z uživatelského rozhraní vybere odpovídající generátor. Uživatel si může zvolit velikost výsledné mapy. Po kliknutí na tlačítko `generate` následně skript změří čas generování hodnot ve funkci `generate()`, vypočte základní statistiky mapy a spustí její vizualizaci po jednotlivých řádcích. Po dokončení vykreslení jsou aktualizovány limity kamery, do mapy jsou náhodně rozmístěny mince a hráč je umístěn na vhodnou průchozí pozici s dostatečně volným okolím.

3.3.1 Náhodné generování místností a chodeb

Implementaci algoritmu najdeme v souboru `room_generator.gd`. Metoda vychází z postupného umísťování náhodně generovaných obdélníkových místností do prázdné mapy. Jednotlivé místnosti jsou následně propojeny chodbami, čímž vzniká souvislá dungeonová struktura. Výsledná mapa je reprezentována jako dvourozměrné pole, ve kterém jsou jednotlivé buňky označeny celočíselnými hodnotami odpovídajícími prázdnému prostoru, podlaze a stěně.

Na začátku funkce `generate(width, height)` je vytvořen lokální generátor náhodných čísel `RandomNumberGenerator`. Použití lokální instance zajišťuje, že generování mapy neovlivňuje jiné části aplikace využívající náhodnost. Následně je inicializována prázdná mapa o zadaných rozměrech, přičemž všechny buňky jsou nastaveny na hodnotu `TILE_EMPTY`. Tím vznikne výchozí reprezentace mapy, do níž budou postupně zapisovány místnosti, chodby a stěny.

Samotné generování místností probíhá v cyklu, který se opakuje do okamžiku, kdy je úspěšně umístěno deset místností, nebo dokud počet pokusů nepřekročí hodnotu `max_attempts`, která je v implementaci nastavena na 100. Každá nová místnost je reprezentována obdélníkem typu `Rect2`. Její šířka a výška jsou generovány náhodně v intervalu od 8 do 16 dlaždic. Pozice levého horního rohu místnosti je následně určena tak, aby místnost zůstala uvnitř hranic mapy a zároveň nebyla umístěna přímo na jejím okraji.

Před přidáním nové místnosti do mapy je provedena kontrola překryvu se všemi dříve umístěnými místnostmi. Tato kontrola je realizována pomocí metody `grow(1).intersects(other)`, což znamená, že se testuje nejen skutečný průnik obdélníků, ale i jejich rozšíření o jednu dlaždici na každou stranu. V praxi tak mezi dvěma místnostmi vždy zůstává alespoň jednopolíčková mezera.

Pokud by se nová místnost překrývala s některou z již existujících, je zahozena a algoritmus pokračuje dalším pokusem.

V případě, že nová místnost překryv nevykazuje, je přidána do seznamu místností a její vnitřní plocha je zapsána do mapy jako podlaha. To je realizováno dvojicí vnořených cyklů, které procházejí všechny buňky uvnitř obdélníku místnosti a nastavují jejich hodnotu na `TILE_FLOOR`. Tím dojde k vyznačení průchozí oblasti odpovídající nové místnosti.

Každá nová místnost je následně propojená chodbou. První místnost je však z tohoto propojování vyloučena. Pro propojení se využívají středy obou místností získané pomocí metody `get_center()`. Vlastní vytvoření chodby zajišťuje pomocná funkce `_carve_corridor()`, která mezi dvěma zadanými body vyznačí chodbu tvaru písmene L. Algoritmus náhodně rozhoduje, zda bude chodba nejprve vedena ve vodorovném a poté ve svislém směru, nebo opačně. Díky tomu výsledné mapy nepůsobí zcela uniformně.

Chodba není tvořena pouze jednou dlaždicí, ale má nastavitelnou šířku. V implementaci je výchozí šířka parametru `corridor_width` nastavena na hodnotu 2. Funkce proto při vykreslování chodby neoznačuje pouze centrální linii, ale také sousední buňky v jejím okolí. Výsledná chodba je tak vizuálně širší a průchod mapou je přehlednější. Při zápisu dlaždic chodby je současně kontrolováno, zda se zapisované souřadnice nacházejí uvnitř hranic mapy, a to pomocí pomocné funkce `_in_bounds()`.

Po dokončení umístování místností a chodeb následuje doplnění stěn. Tuto operaci provádí funkce `_add_walls()`, která prochází všechny buňky mapy a u každé dlaždice podlahy kontroluje její osmibuněčné okolí. Pokud se v sousedství nachází buňka označená jako prázdný prostor, je její hodnota změněna na `TILE_WALL`. Tímto způsobem vznikne souvislé ohraničení všech průchozích částí mapy bez nutnosti explicitně generovat stěny již během vytváření místností a chodeb.

Výsledkem celé funkce je dvourozměrné pole obsahující kompletní rozložení mapy. Tato reprezentace je následně předána dalším částem aplikace, které zajišťují výpočet statistik a vizualizaci mapy v herním prostředí.

3.3.2 Binary Space Partitioning

Implementace metody BSP se nachází v souboru `bsp_generator.gd`. Skript využívá rekursivní rozdělování prostoru mapy na menší oblasti pomocí binární stromové struktury. Každý uzel stromu reprezentuje obdélníkovou oblast mapy, která může být dále rozdělena na dvě pod-oblasti.

Generování začíná vytvořením počáteční oblasti odpovídající celé velikosti mapy. Tato oblast představuje kořen stromu a je postupně rozdělena na dvě části. Směr dělení je určován podle poměru šířky a výšky oblasti. Pokud je oblast výrazně širší než vyšší, je preferováno vertikální dělení, v opačném případě horizontální. Pokud jsou rozměry oblasti podobné, je směr dělení zvolen náhodně.

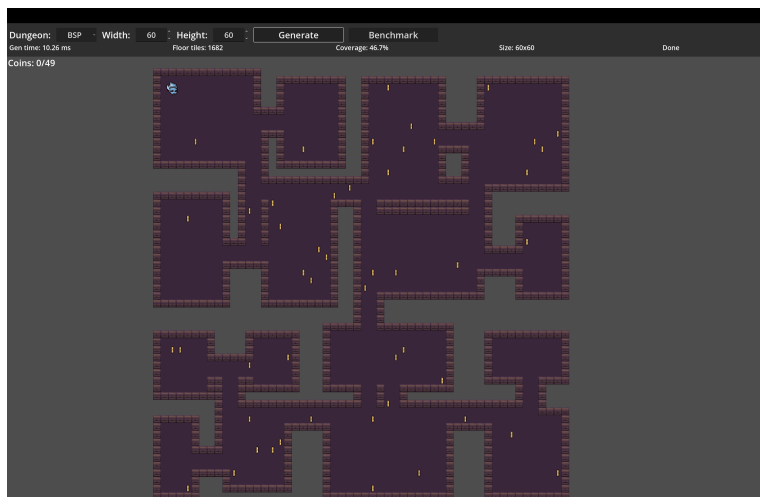
Při každém dělení je zvolena pozice řezu tak, aby vzniklé podoblasti splňovaly minimální požadovanou velikost. Tím je zajištěno, že v nich bude možné

vytvořit místnost. Každé rozdělení vytváří dva nové uzly stromu, které jsou dále zpracovávány stejným způsobem.

Rekurzivní dělení pokračuje do okamžiku, kdy další rozdělení již není možné kvůli minimální velikosti oblasti. Takové uzly představují listy stromu. V každém listu je následně vytvořena místnost, jejíž velikost i pozice jsou generovány náhodně uvnitř příslušné oblasti.

Po vytvoření místností jsou jednotlivé části stromu postupně propojeny chodbami. Propojení probíhá při návratu z rekurzivního volání, kdy jsou vybrány vhodné body v místnostech patřících do dvou sousedních oblastí. Mezi těmito body je vytvořena chodba zajišťující průchod mezi jednotlivými částmi mapy.

Výsledná mapa je zapsána do dvourozměrného pole, které obsahuje informace o typu jednotlivých dlaždic. Průchozí části odpovídají podlahám místností a chodeb, zatímco okolní prostor je následně doplněn o stěny.



Obrázek 3: Ukázka mapy generované pomocí algoritmu BSP

3.3.3 Metoda náhodné procházky

Náhodná procházka je implementována v souboru `drunken_generator.gd`. Generování mapy probíhá opět ve funkci `generate(width, height)`, která vytváří dvourozměrné pole reprezentující výslednou mapu.

Na začátku funkce je vytvořena prázdná mapa o zadaných rozměrech, ve které jsou všechny buňky inicializovány jako neprůchozí prostor. Následně je vypočten cílový počet průchozích dlaždic, který je určen jako určitý podíl z celkové plochy mapy. Tento parametr určuje, jak velkou část mapy bude algoritmus postupně odkrývat.

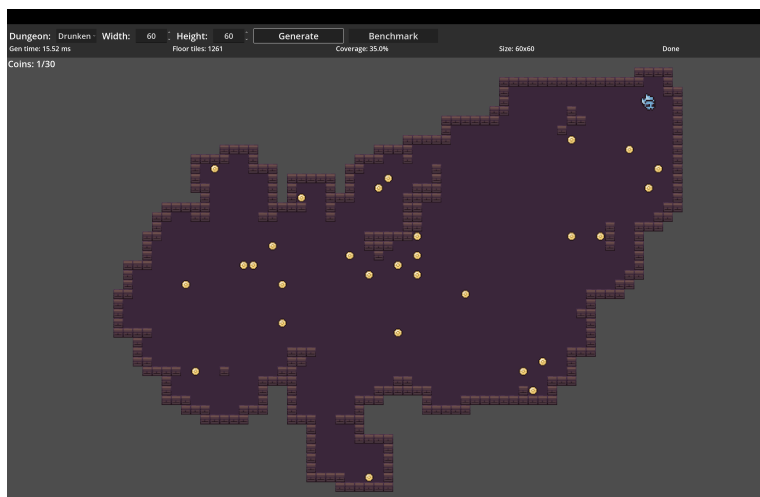
Samotné generování probíhá pomocí simulace náhodného pohybu po mapě. Algoritmus začíná na náhodně zvolené pozici uvnitř mapy, která je označena jako průchozí dlaždice. V každém kroku je následně náhodně zvolen směr pohybu, typicky nahoru, dolů, doleva nebo doprava. Pokud se nová pozice nachází uvnitř

hranic mapy, je tato buňka označena jako podlaha a aktuální pozice generátoru se přesune na tuto dlaždici.

Tento proces se opakuje, dokud počet vytvořených průchozích dlaždic nedosáhne předem definovaného cílového počtu. V důsledku náhodného pohybu vzniká nepravidelná struktura připomínající přirozeně vzniklé jeskynní prostory. Na rozdíl od metod založených na explicitním generování místností zde nevznikají jasně definované obdélníkové prostory, ale spíše organicky tvarované oblasti.

Po dokončení samotné náhodné procházky jsou kolem průchozích dlaždic doplněny stěny. Tento krok zajišťuje, že prázdné buňky sousedící s podlahou jsou označeny jako stěny, čímž vznikne souvislé ohraničení průchozí části mapy.

Výsledkem funkce je dvourozměrné pole obsahující rozložení jednotlivých typů dlaždic mapy. Vzhledem k tomu, že cílový počet průchozích dlaždic je určen předem, je výsledné procento pokrytí mapy průchozím prostorem pro danou velikost mapy poměrně stabilní. Variabilita generovaných map se proto projevuje především ve tvaru a rozmístění vytvořených prostorů.



Obrázek 4: Ukázka mapy generované pomocí algoritmu náhodné procházky

3.3.4 Buněčné automaty

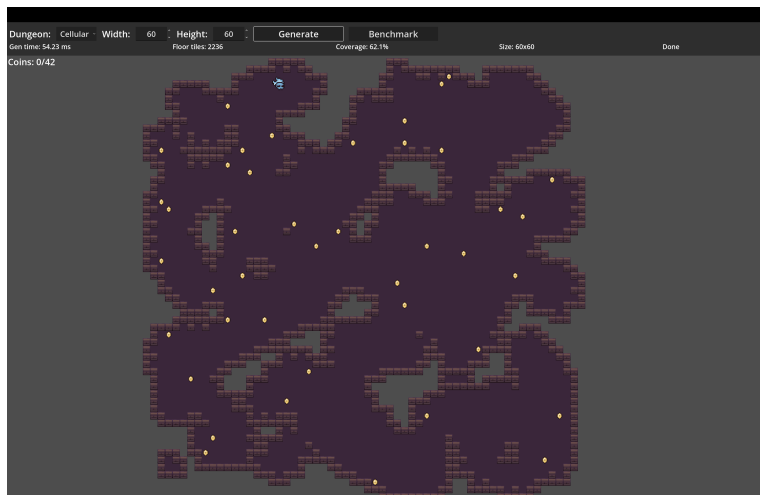
Implementace buněčných automatů je v souboru `cellular_generator.gd`. První fáze vytvoří počáteční mapu náhodným vyplněním buněk podlahou podle pravděpodobnosti `FILL_PROBABILITY`. Okraje mapy zůstávají prázdné, aby se předešlo problémům při následném vyhodnocování sousedství.

V další fázi proběhne několik simulačních kroků. V každém kroku se vytváří nová mapa, aby se průběžné změny nepromítaly do výpočtu sousedů ve stejném kroku. Pro každou buňku se spočítá počet podlahových sousedů v Mooreově okolí a podle prahové hodnoty se rozhodne o novém stavu buňky. Opakováním těchto kroků se náhodný šum postupně vyhladí do organičtějších tvarů.

Po dokončení simulace je nad výslednou podlahou doplněna vrstva stěn. Výstupem jsou nepravidelné mapy jeskynního charakteru, které jsou vhodné pro

porovnání s metodami generujícími pravidelnější struktury.

U map generovaných metodou buněčných automatů se mohou vyskytovat izolované průchozí oblasti. Z tohoto důvodu bylo při umísťování hráče nutné zohlednit souvislost průchozí plochy a upřednostnit umístění do největší spojitě oblasti mapy.



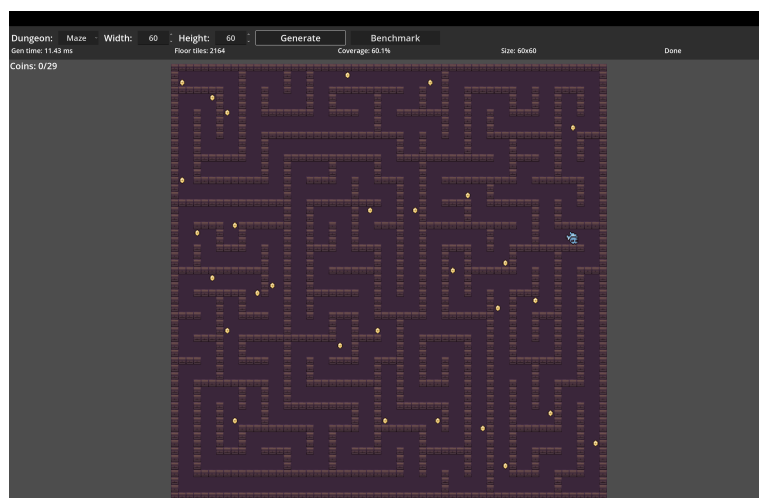
Obrázek 5: Ukázka mapy generované pomocí algoritmu buněčných automatů

3.3.5 Metoda bludiště

Metoda bludiště je implementována v souboru `maze_generator.gd` a vychází z algoritmu recursive backtracker (DFS se zásobníkem). Vnitřní reprezentace není tvořena přímo dlaždicemi mapy, ale mřížkou abstraktních buněk, z nichž každá obsahuje bitovou masku čtyř stěn.

Generování začíná v počáteční buňce a postupně odstraňuje stěny mezi aktuální buňkou a náhodně vybraným nenavštíveným sousedem. Pokud aktuální buňka nemá dostupného nenavštíveného souseda, algoritmus se vrací zpět po zásobníku. Tento postup pokračuje, dokud nejsou navštíveny všechny buňky.

Po dokončení průchodu je buněčná reprezentace převedena do tile mapy. Každá buňka je vykreslena jako menší průchozí oblast a podle odstraněných stěn se mezi sousedními buňkami vytvářejí koridory. V závěru jsou stejně jako u ostatních metod doplněny stěny kolem podlahy.



Obrázek 6: Ukázka mapy generované pomocí algoritmu bludiště

4 Výsledky

4.1 Porovnání výsledných map

Algoritmus generování bludiště byl do implementace zahrnut především pro účely demonstrace rozdílných přístupů k procedurálnímu generování map. Výsledky však ukazují, že tento typ algoritmu vytváří velmi pravidelnou strukturu mapy s vysokou propojeností, která odpovídá klasickému bludišti.

Pro generování dungeonových map, které obvykle obsahují výrazněji oddělené místnosti a různé typy prostorů, je tento přístup méně vhodný. Výsledná prostředí mohou působit monotónně a neposkytují dostatečnou variabilitu struktury.

4.2 Výpočetní náročnost metod

4.3 Praktické využití výsledků

5 Diskuse

5.1 Silné a slabé stránky jednotlivých metod

5.2 Zkušenosti z implementace

5.3 Možnosti dalšího rozšíření práce

Závěr

Conclusions

A Ukázky zdrojového kódu

B Odkaz na repozitář projektu

C Zkratky

PCG - Procedural Content Generation RAM - Random Access Memory BSP
- Binary Space Partitioning ASCII - American Standard Code for Information
Interchange

D Obsah elektronických dat

Povinné položky struktury dat jsou:

text/

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

README.*

Textový soubor (s příponou např. `.txt`) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,

- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwareového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovolují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru README.*.

Literatura

- [1] Procedural Content Generation Wiki. *Rogue* [online]. 2016 [cit. 2024-5-22]. Dostupný z: <http://pcg.wikidot.com/pcg-games:rogue>.
- [2] Wikipedia Contributors. *Rogue (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 2026]. Dostupný z: [https://en.wikipedia.org/w/index.php?title=Rogue_\(video_game\)&oldid=1337873899](https://en.wikipedia.org/w/index.php?title=Rogue_(video_game)&oldid=1337873899).
- [3] Wikipedia Contributors. *Elite (video game)* — *Wikipedia, The Free Encyclopedia*. 2026. [Online; accessed 22-February-2026]. Dostupný z: [https://en.wikipedia.org/wiki/Elite_\(video_game\)](https://en.wikipedia.org/wiki/Elite_(video_game)).
- [4] Linden, Roland; Lopes, Ricardo; Bidarra, Rafael. Procedural Generation of Dungeons. *Computational Intelligence and AI in Games, IEEE Transactions on*. 2014, roč. 6, s. 78–89. Dostupný také z: <http://dx.doi.org/10.1109/TCIAIG.2013.2290371>.
- [5] Shaker, Noor; Togelius, Julian; Nelson, Mark J. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. 2016. 247 s. ISBN 978-3-319-42714-0.
- [6] Monaghan, Zachary. *Comparing Procedural Content Generation Algorithms for Dungeon Generation*. 2019. [Online; accessed 2026]. Dostupný také z: <https://arrow.tudublin.ie/cgi/viewcontent.cgi?article=1189&context=scschcomdis>.
- [7] Schuller, Dan. *Dive into beneath Apple Manor: A pre-rogue roguelike*. 2021. Dostupný z: <https://howtomakeanrpg.com/r/l/g/apple-manor.html>.
- [8] Wikipedia Contributors. *Beneath Apple Manor* — *Wikipedia, The Free Encyclopedia*. 2024. Dostupný z: https://en.wikipedia.org/w/index.php?title=Beneath_Apple_Manor&oldid=1260179812%22.
- [9] *Godot Engine Documentation*. 2024. [Online; accessed 2026]. Dostupný z: <https://docs.godotengine.org>.
- [10] *GDScript Reference*. 2024. [Online; accessed 2026]. Dostupný z: <https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/>.